1 Mathematics 1
2 Combinatorial 1
3 Number theory 2
4 Data structures 4
5 Strings 5

Mathematics (1)

1.1 Sums

$$c^a + c^{a+1} + \cdots + c^b = \frac{c^{b+1} - c^a}{c-1}, c \neq 1$$

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$
$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(2n+1)(n+1)}{6}$$
$$1^3 + 2^3 + 3^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$$
$$1^4 + 2^4 + 3^4 + \cdots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

1.2 Probability theory

Let X be a discrete random variable with probability $p_X(x)$ of assuming the value x. It will then have an expected value (mean) $\mu = \mathbb{E}(X) = \sum_x x p_X(x)$ and variance $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$ where σ is the standard deviation. If X is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.

Expectation is linear:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

For independent X and Y,

$$V(aX + bY) = a^2V(X) + b^2V(Y).$$

1.2.1 Discrete distributions

Binomial distribution

The number of successes in n independent yes/no experiments, each which yields success with probability p is $\mathrm{Bin}(n, p)$, $n = 1, 2, \ldots, 0 \leq p \leq 1$.

$$p(k) = \binom{n}{k} p^k (1-p)^{n-k}$$

$$\mu = np,\ \sigma^2 = np(1-p)$$

$\mathrm{Bin}(n, p)$ is approximately $\mathrm{Po}(np)$ for small p.

First success distribution

The number of trials needed to get the first success in independent yes/no experiments, each which yields success with probability p is $\mathrm{Fs}(p)$, $0 \leq p \leq 1$.

$$p(k) = p(1-p)^{k-1},\ k = 1, 2, \ldots$$

$$\mu = \frac{1}{p},\ \sigma^2 = \frac{1-p}{p^2}$$

1.3 Floor sum

floorSum.h
Description: //todo
Time: $\mathcal{O}(log(a))$

d41d8c, 9 lines

```
long long floorSum(long long a, long long b, long long c, long
     long n){
    if(!a) return (b / c) * (n + 1);
    if(a >= c or b >= c) {
        return ((n * (n + 1)) / 2) * (a / c) + (n + 1)
                * (b / c) + floorSum(a % c, b % c, c, n);
    }
    long long m = (a * n + b) / c;
    return m * n - floorSum(c, c - b - 1, a, m - 1);
}
```

For non-negative integers a, b, n and $c > 0$, define

$$F(a, b, c, n) = \sum_{i=0}^{n} \left\lfloor \frac{ai + b}{c} \right\rfloor.$$

Let $T_n = \frac{n(n+1)}{2}$. If $a \geq c$ or $b \geq c$,

$$F(a, b, c, n) = T_n \left\lfloor \frac{a}{c} \right\rfloor + (n+1) \left\lfloor \frac{b}{c} \right\rfloor + F(a \bmod c, b \bmod c, c, n).$$

Otherwise, let $m = \lfloor \frac{an+b}{c} \rfloor$. Then

$$F(a, b, c, n) = \begin{cases} 0 & m = 0, \\ mn - F(c, c-b-1, a, m-1) & m > 0. \end{cases}$$

The recursion is based on the Euclidean algorithm and runs in $\mathcal{O}(\log \max(a, c))$ time.

1.3.1 Sum of $\lfloor n/i \rfloor$

To compute

$$S(n) = \sum_{i=1}^{n} \left\lfloor \frac{n}{i} \right\rfloor,$$

group consecutive indices with equal quotients. For each l, let

$$q = \left\lfloor \frac{n}{l} \right\rfloor, \qquad r = \left\lfloor \frac{n}{q} \right\rfloor.$$

Then $\lfloor n/i \rfloor = q$ for all $l \leq i \leq r$, so add $q(r - l + 1)$ and continue from $l = r + 1$. The algorithm runs in $\mathcal{O}(\sqrt{n})$ time.

1.4 Linear recurrences

For a recurrence

$$f_n = c_1 f_{n-1} + \cdots + c_k f_{n-k},$$

define the state vector

$$v_n = (f_n, f_{n-1}, \ldots, f_{n-k+1})^T.$$

Then $v_n = M v_{n-1}$, where

$$M = \begin{pmatrix} c_1 & c_2 & \ldots & c_{k-1} & c_k \\ 1 & 0 & \ldots & 0 & 0 \\ 0 & 1 & \ldots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \ldots & 1 & 0 \end{pmatrix}.$$

The first row computes the next term; the remaining rows shift the previous terms. Thus,

$$v_n = M^{n-k+1} v_{k-1},$$

where $v_{k-1} = (f_{k-1}, \ldots, f_0)^T$.

Combinatorial (2)

2.1 Permutations

2.1.1 Factorial

IntPerm.h
Description: Permutation -> integer conversion. (Not order preserving.) Integer -> permutation can use a lookup table.
Time: $\mathcal{O}(n)$

d41d8c, 6 lines

```
int permToInt(vector<int>& v) {
  int use = 0, i = 0, r = 0;
  for(int x:v) r = r * ++i + __builtin_popcount(use & -(1<<x)),
    use |= 1 << x;                    // (note: minus, not ~!)
  return r;
}
```

2.1.2 Cycles

Let $g_S(n)$ be the number of n-permutations whose cycle lengths all belong to the set S. Then

$$\sum_{n=0}^{\infty} g_S(n) \frac{x^n}{n!} = \exp\left(\sum_{n \in S} \frac{x^n}{n} \right)$$

2.1.3 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1) + D(n-2)) = nD(n-1) + (-1)^n = \left\lfloor \frac{n!}{e} \right\rceil$$

2.1.4 Burnside's lemma

Given a group G of symmetries and a set X, the number of elements of X *up to symmetry* equals

$$\frac{1}{|G|}\sum_{g\in G}|X^g|,$$

where X^g are the elements fixed by g ($g.x = x$).

If $f(n)$ counts "configurations" (of some sort) of length n, we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n}\sum_{k=0}^{n-1} f(\gcd(n,k)) = \frac{1}{n}\sum_{k|n} f(k)\phi(n/k).$$

2.2 Partitions and subsets

2.2.1 Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1,\ p(n) = \sum_{k\in\mathbb{Z}\setminus\{0\}} (-1)^{k+1} p(n - k(3k-1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	∼2e5	∼2e8

2.2.2 Lucas' Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_k p^k + \ldots + n_1 p + n_0$ and $m = m_k p^k + \ldots + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^{k} \binom{n_i}{m_i} \pmod p$.

2.2.3 Binomials

multinomial.h

Description: Computes $\binom{k_1+\cdots+k_n}{k_1,k_2,\ldots,k_n} = \frac{(\sum k_i)!}{k_1!k_2!\ldots k_n!}$. d41d8c, 7 lines

```
long long multinomial(vector<int>& v) {
  long long c = 1, m = v.empty() ? 1 : v[0];
  for (int i = 1; i < (int)v.size; ++i)
    for (int j = 0; j < v[i]; ++j)
      c = c * ++m / (j + 1);
  return c;
}
```

2.3 General purpose numbers

2.3.1 Stars and bars

The number of non-negative integer solutions to $x_1 + \cdots + x_k = n$ is

$$\binom{n+k-1}{k-1}.$$

For positive integer solutions, the number is

$$\binom{n-1}{k-1}.$$

2.3.2 Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$$c(n,k) = c(n-1,k-1) + (n-1)c(n-1,k),\ c(0,0) = 1$$
$$\textstyle\sum_{k=0}^{n} c(n,k)x^k = x(x+1)\ldots(x+n-1)$$

2.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k j:s s.t. $\pi(j) > \pi(j+1)$, $k+1$ j:s s.t. $\pi(j) \geq j$, k j:s s.t. $\pi(j) > j$.

$$E(n,k) = (n-k)E(n-1,k-1) + (k+1)E(n-1,k)$$

$$E(n,0) = E(n,n-1) = 1$$

$$E(n,k) = \sum_{j=0}^{k} (-1)^j \binom{n+1}{j} (k+1-j)^n$$

2.3.4 Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n,k) = S(n-1,k-1) + kS(n-1,k)$$

$$S(n,1) = S(n,n) = 1$$

$$S(n,k) = \frac{1}{k!}\sum_{j=0}^{k} (-1)^{k-j} \binom{k}{j} j^n$$

2.3.5 Labeled unrooted trees

on n vertices: n^{n-2}
on k existing trees of size n_i: $n_1 n_2 \cdots n_k n^{k-2}$
with degrees d_i: $(n-2)!/((d_1-1)!\cdots(d_n-1)!)$

2.3.6 Catalan numbers

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1,\ C_{n+1} = \frac{2(2n+1)}{n+2}C_n,\ C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}.$$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with with $n+1$ leaves (0 or 2 children).
- ordered trees with $n+1$ vertices.
- ways a convex polygon with $n+2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

Number theory (3)

3.1 Modular arithmetic

ModularArithmetic.h

Description: Operators for modular arithmetic. d41d8c, 24 lines

```
template<ll MOD> // using Mint = Mod<998244353>;
struct Mod {
    ll x;
    Mod(ll y = 0) : x((y %= MOD) < 0 ? y + MOD : y) {}
    Mod(ll y, int) : x(y) {}
    Mod operator+(Mod b) { return {x + b.x, 0}; }
    Mod operator-(Mod b) { return {x - b.x + MOD, 0}; }
    Mod operator*(Mod b) { return {x * b.x % MOD, 0}; }
    Mod operator/(Mod b) { return *this * b.inv(); }
    Mod inv() {
        ll a = x, b = MOD, u = 1, v = 0;
        while (b) {
            ll t = a / b;
            a -= t * b; swap(a, b);
            u -= t * v; swap(u, v);
        }
        assert(a == 1); return u;
    }
    Mod pow(ll e) {
        assert(e >= 0); Mod a = *this, r = 1;
        for (; e; e >>= 1, a = a * a) if (e & 1) r = r * a;
        return r;
    }
};
```

ModInverse.h

Description: Assumes LIM $\leq$ mod and that mod is a prime. d41d8c, 4 lines

```
const ll mod = 1e9 + 7, LIM = 200000;
ll inv[LIM]; inv[1] = 1;
for (int i = 2; i < LIM; ++i)
    inv[i] = mod - (mod / i) * inv[mod % i] % mod;
```

ModPow.h d41d8c, 7 lines

```
const ll mod = 1e9 + 7;
ll bpow(ll a, ll b) {
  ll ans = 1;
  for (; b; a = a * a % mod, b >>= 1)
    if (b & 1) ans = ans * a % mod;
  return ans;
}
```

ModLog.h

Description: Returns the smallest $x > 0$ s.t. $a^x = b \pmod m$, or -1 if no such x exists. modLog(a,1,m) can be used to calculate the order of a.
Time: $\mathcal{O}(\sqrt{m})$ d41d8c, 13 lines

```
ll modLog(ll a, ll b, ll m) {
  ll n = (ll)sqrt(m) + 1;
  ll e = 1, f = 1, j = 1;
  unordered_map<ll, ll> A;
  while (j <= n && (e = f = e * a % m) != b % m)
    A[e * b % m] = j++;
  if (e == b % m) return j;
  if (__gcd(m, e) == __gcd(m, b))
    for (int i = 2; i < n + 2; ++i)
      if (A.count(e = e * f % m))
        return n * i - A[e];
  return -1;
}
```

ModSum.h

Description: Sums of mod'ed arithmetic progressions.
`modsum(to, c, k, m)` $= \sum_{i=0}^{to-1} (ki + c)\%m$. `divsum` is similar but for floored division.
Time: $\log(m)$, with a large constant.

d41d8c, 15 lines

```
ull sumsq(ull to) { return to / 2 * ((to-1) | 1); }

ull divsum(ull to, ull c, ull k, ull m) {
  ull res = k / m * sumsq(to) + c / m * to;
  k %= m; c %= m;
  if (!k) return res;
  ull to2 = (to * k + c) / m;
  return res + (to - 1) * to2 - divsum(to2, m - 1 - c, m, k);
}

ll modsum(ull to, ll c, ll k, ll m) {
  c = ((c % m) + m) % m;
  k = ((k % m) + m) % m;
  return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}
```

ModMulLL.h

Description: Calculate $a \cdot b \bmod c$ (or $a^b \bmod c$) for $0 \le a, b \le c \le 7.2 \cdot 10^{18}$.
Time: $\mathcal{O}(1)$ for `modmul`, $\mathcal{O}(\log b)$ for `modpow`

d41d8c, 3 lines

```
ull modmul(ull a, ull b, ull M) {
  return (unsigned __int128)a * b % M;
}
```

ModSqrt.h

Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 = a \pmod p$ ($-x$ gives the other solution).
Time: $\mathcal{O}\left(\log^2 p\right)$ worst case, $\mathcal{O}(\log p)$ for most p

"ModPow.h" d41d8c, 23 lines

```
ll sqrt(ll a, ll p) {
  a %= p; if (a < 0) a += p;
  if (a == 0) return 0;
  assert(bpow(a, (p - 1) / 2, p) == 1); // else no solution
  if (p % 4 == 3) return bpow(a, (p + 1) / 4, p);
  ll s = p - 1, n = 2;
  int r = 0, m;
  while (s % 2 == 0)
    ++r, s /= 2;
  while (bpow(n, (p - 1) / 2, p) != p - 1) ++n;
  ll x = bpow(a, (s + 1) / 2, p);
  ll b = bpow(a, s, p), g = bpow(n, s, p);
  for (; ; r = m) {
    ll t = b;
    for (m = 0; m < r && t != 1; ++m)
      t = t * t % p;
    if (m == 0) return x;
    ll gs = bpow(g, 1ll << (r - m - 1), p);
    g = gs * gs % p;
    x = x * gs % p;
    b = b * g % p;
  }
}
```

3.2 Primality

FastEratosthenes.h

Description: Prime sieve for generating all primes smaller than N.
Time: ??

d41d8c, 11 lines

```
const int N = 1e7;

int lp[N]; vector<int> pr;
for (int i = 2; i < N; ++i) {
    if (lp[i] == 0)
        lp[i] = i; pr.push_back(i);
    for (int j = 0; i * 1ll * pr[j] < N; ++j) {
        lp[i * pr[j]] = pr[j];
        if (pr[j] == lp[i]) break;
    }
}
```

MillerRabin.h

Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.
Time: 7 times the complexity of $a^b \mod c$.

"ModMulLL.h" d41d8c, 12 lines

```
bool isPrime(ull n) {
  if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
  ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
      s = __builtin_ctzll(n-1), d = n >> s;
  for (ull a : A) {   // ^ count trailing zeroes
    ull p = bpow(a % n, d, n), i = s;
    while (p != 1 && p != n - 1 && a % n && i--)
      p = modmul(p, p, n);
    if (p != n - 1 && i != s) return 0;
  }
  return 1;
}
```

Factor.h

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).
Time: $\mathcal{O}\left(n^{1/4}\right)$, less for numbers with small factors.

"ModMulLL.h", "MillerRabin.h" d41d8c, 19 lines

```
ull pollard(ull n) {
  ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
  auto f = [&](ull x) { return modmul(x, x, n) + i; };
  while (t++ % 40 || __gcd(prd, n) == 1) {
    if (x == y) x = ++i, y = f(x);
    if ((q = modmul(prd, max(x, y) - min(x, y), n))) prd = q;
    x = f(x), y = f(f(y));
  }
  return __gcd(prd, n);
}

vector<ull> factor(ull n) {
  if (n == 1) return {};
  if (isPrime(n)) return {n};
  ull x = pollard(n);
  auto l = factor(x), r = factor(n / x);
  l.insert(l.end(), all(r));
  return l;
}
```

3.3 Divisibility

euclid.h

Description: Finds two integers x and y, such that $ax + by = \gcd(a, b)$. If you just need gcd, use the built in __gcd instead. If a and b are coprime, then x is the inverse of $a \pmod b$.

d41d8c, 5 lines

```
ll euclid(ll a, ll b, ll &x, ll &y) {
  if (!b) return x = 1, y = 0, a;
  ll d = euclid(b, a % b, y, x);
  return y -= a / b * x, d;
}
```

CRT.h

Description: Chinese Remainder Theorem.
`crt(a, m, b, n)` computes x such that $x \equiv a \pmod m$, $x \equiv b \pmod n$. If $|a| < m$ and $|b| < n$, x will obey $0 \le x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.
Time: $\log(n)$

"euclid.h" d41d8c, 7 lines

```
ll crt(ll a, ll m, ll b, ll n) {
  if (n > m) swap(a, b), swap(m, n);
  ll x, y, g = euclid(m, n, x, y);
  assert((a - b) % g == 0); // else no solution
  x = (b - a) % n * x % n / g * m + a;
  return x < 0 ? x + m * n/g : x;
}
```

3.3.1 Bézout's identity

For $a \neq, b \neq 0$, then $d = gcd(a, b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If (x, y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

phiFunction.h

Description: *Euler's* ϕ function is defined as $\phi(n) :=$ # of positive integers $\le n$ that are coprime with n. $\phi(1) = 1$, p prime $\Rightarrow \phi(p^k) = (p-1)p^{k-1}$, m, n coprime $\Rightarrow \phi(mn) = \phi(m)\phi(n)$. If $n = p_1^{k_1} p_2^{k_2} ... p_r^{k_r}$ then $\phi(n) = (p_1 - 1)p_1^{k_1 - 1} ... (p_r - 1)p_r^{k_r - 1}$. $\phi(n) = n \cdot \prod_{p|n}(1 - 1/p)$.
$\sum_{d|n} \phi(d) = n$, $\sum_{1 \le k \le n, \gcd(k,n)=1} k = n\phi(n)/2, n > 1$
Euler's thm: a, n coprime $\Rightarrow a^{\phi(n)} \equiv 1 \pmod n$.
Fermat's little thm: p prime $\Rightarrow a^{p-1} \equiv 1 \pmod p \ \forall a$.

d41d8c, 7 lines

```
int phi[N];
void calculatePhi() {
  for (int i = 0; i < N; ++i)
    phi[i] = i & 1 ? i : i / 2;
  for (int i = 3; i < N; i += 2) if(phi[i] == i)
    for (int j = i; j < N; j += i) phi[j] -= phi[j] / i;
}
```

3.4 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2), \ \ b = k \cdot (2mn), \ \ c = k \cdot (m^2 + n^2),$$

with $m > n > 0$, $k > 0$, $m \perp n$, and either m or n even.

3.5 Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a, except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^{\times}$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

3.6 Estimates

N	n	$\max_{n \le N} \tau(n)$
10^3	840	32
10^6	720720	240
10^9	735134400	1344
10^{12}	963761198400	6720
10^{15}	866421317361600	26880
10^{18}	897612484786617600	103680

3.7 Mobius Function

Mobius.h

d41d8c, 20 lines

```
const int N = 1e6 + 5;
ll mob[N]; bool prime[N];
void sieve(){
    for(int i = 0; i < N; i++){
        mob[i] = 1;
        prime[i] = 1;
    }

    mob[1] = 1;
    prime[0] = prime[1] = 0;
    for(int i = 2; i < N; i++){
        if(prime[i]){
            for(int j = i; j < N; j += i){
                if(j > i) prime[j] = 0;
                if((j / i) % i == 0) mob[j] = 0;
                mob[j] *= -1;
            }
        }
    }
}
```

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d) g(n/d)$$

Other useful formulas/forms:

$\sum_{d|n} \mu(d) = [n = 1]$ (very useful)

$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n) g(d)$

$g(n) = \sum_{1 \le m \le n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \le m \le n} \mu(m) g(\lfloor \frac{n}{m} \rfloor)$

Data structures (4)

OrderStatisticTree.h

d41d8c, 5 lines

```
#include <bits/extc++.h>
using namespace __gnu_pbds;

template<typename T>
using oset = tree<T, null_type, less<T>, rb_tree_tag,
     tree_order_statistics_node_update>;
```

HashMap.h

Description: Hash map with mostly the same API as unordered_map, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided).

d41d8c, 8 lines

```
#include <bits/extc++.h>
using namespace __gnu_pbds;
// To use most bits rather than just the lowest ones:
struct chash { // large odd number for C
  const uint64_t C = ll(4e18 * acos(0)) | 71;
  ll operator()(ll x) const { return __builtin_bswap64(x*C); }
};
gp_hash_table<ll,int,chash> h({},{},{},{},{1<<16});
```

IterativeSeg.h

d41d8c, 21 lines

```
int t[2 * N];
void build(int n) {
  for (int i = 0; i < n; ++i)
    t[n + i] = a[i];
  for (int i = n - 1; i > 0; --i)
    t[i] = t[i << 1] + t[i << 1 | 1];
}

void modify(int pos, int val) {  // a[pos] := val
  for (t[pos += n] = val; pos > 1; pos >>= 1)
    t[pos >> 1] = t[pos] + t[pos ^ 1];
}

int query(int l, int r) {  // [l, r)
  int res = 0;
  for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
    if (l & 1) res += t[l++];
    if (r & 1) res += t[--r];
  }
  return res;
}
```

SparseTable.h

d41d8c, 14 lines

```
int st[N][LG];
void build() {
  for (int i = 0; i < n; ++i) st[i][0] = a[i];
  for (int j = 1; j < LG; ++j) {
    for (int i = 0; i + (1 << j) <= n; ++i) {
      st[i][j] = max(st[i][j - 1], st[i + (1 << (j - 1))][j -
          1]);
    }
  }
}

int get(int l, int r) {
  int lg = __lg(r - l + 1);
  return max(st[l][lg], st[r - (1 << lg) + 1][lg]);
}
```

UnionFindRollback.h

d41d8c, 23 lines

```
struct DSU {
  vector<int> par, sz;
  vector<pair<int*, int>> history;

  DSU(int n) : par(n, -1), sz(n, 1) {}
  int time() { return (int)history.size(); }
  int get(int v) { return par[v] < 0 ? v : get(par[v]); }
  void rollback(int t) {
    while ((int)st.size() > t) {
      *history.back().first = history.back().second;
      history.pop_back();
    }
  }
  void join(int a, int b) {
    a = get(a), b = get(b);
    if (a == b) return;
    if (sz[a] < sz[b]) swap(a, b);
    history.push_back({&sz[a], sz[a]});
    history.push_back({&par[b], par[b]});
    sz[a] += sz[b];
    par[b] = a;
  }
}
```

ConvexHullTrick.h

d41d8c, 31 lines

```
struct line {
  long long k, b;
  long double get(long double x) {
    return k * x + b;
  }
};
long double ist(line a, line b) {
  return (long double)(b.b - a.b) / (a.k - b.k);
}

vector<line> cht;
vector<long double> pt;

void add(line x) {
  while (cht.size() >= 2 && cht.back().get(pt.back()) > x.get(
      pt.back())) {
    cht.pop_back(); pt.pop_back();
  }

  if (cht.empty()) {
    pt.push_back(-inf);
  } else {
    pt.push_back(ist(cht.back(), x));
  }

  cht.push_back(x);
};

long double get(long long x) {
  int it = upper_bound(pt.begin(), pt.end(), x) - pt.begin() -
      1;
  return cht[it].get(x);
};
```

LiChaoTree.h

d41d8c, 45 lines

```
struct line {
  long long k, b;
  long long get(long long x) {
    return k * x + b;
  }
};

struct {
  line t[16 * N];
  int l[16 * N], r[16 * N];
  int root = 1, last = 2;

  void init(int v) {
    t[v].k = 0; t[v].b = 1e18; l[v] = r[v] = 0;
  }
  void update(int v, long long tl, long long tr, line x) {
    long long tm = (tl + tr) >> 1;

    if (t[v].get(tm) > x.get(tm)) {
```

```
      swap(t[v], x);
    }

    if (t[v].get(tl) > x.get(tl)) {
      if (!l[v]) l[v] = last++, init(l[v]);
      update(l[v], tl, tm, x);
    } else if (t[v].get(tr) > x.get(tr)) {
      if (!r[v]) r[v] = last++, init(r[v]);
      update(r[v], tm + 1, tr, x);
    }
  }
  long long get(int v, long long tl, long long tr, long long
       pos) {
    if (tl == tr) {
      return t[v].get(pos);
    }

    long long tm = (tl + tr) >> 1;
    if (pos <= tm) {
      if (!l[v]) return t[v].get(pos);
      return min(t[v].get(pos), get(l[v], tl, tm, pos));
    } else {
      if (!r[v]) return t[v].get(pos);
      return min(t[v].get(pos), get(r[v], tm + 1, tr, pos));
    }
  }
} T;
```

LineContainer.h

d41d8c, 30 lines

```
struct Line {
  mutable ll k, m, p;
  bool operator<(const Line& o) const { return k < o.k; }
  bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
  // (for doubles, use inf = 1/.0, div(a,b) = a/b)
  static const ll inf = LLONG_MAX;
  ll div(ll a, ll b) { // floored division
    return a / b - ((a ^ b) < 0 && a % b); }
  bool isect(iterator x, iterator y) {
    if (y == end()) return x->p = inf, 0;
    if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
    else x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;
  }
  void add(ll k, ll m) {
    auto z = insert({k, m, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
    while ((y = x) != begin() && (--x)->p >= y->p)
      isect(x, erase(y));
  }
  ll query(ll x) {
    assert(!empty());
    auto l = *lower_bound(x);
    return l.k * x + l.m;
  }
};
```

FenwickTree.h

d41d8c, 15 lines

```
struct F {
  vector<long long> t;
  F(int n) : t(n) {}
  void update(int l, long long val) { // a[l] += val
    for (; l < (int)t.size(); l |= l + 1) t[l] += val;
  }
  long long query(int r) { // sum of values in [0, r]
    long long res = 0;
    for (; r >= 0; r = (r & (r + 1)) - 1) res += t[r];
    return res;
  }
  long long query(int l, int r) { // sum of values in [l, r]
    return query(r) - query(l - 1);
  }
};
```

FenwickTreeRQ.h

d41d8c, 25 lines

```
struct F {
  int n; vector<long long> t1, t2;
  F(int m) : t1(m), t2(m), n(m) {}
  void update(int l, long long x) {
    if (l < 0) return;
    for (int i = l; i >= 0; i = (i & (i + 1)) - 1)
      t2[i] += x;
    for (int i = l | (l + 1); i < n; i |= i + 1)
      t1[i] += x * (l - (i & (i + 1)) + 1);
  }
  long long get(int r) {
    if (r < 0) return 0; long long res = 0;
    for (int i = r; i >= 0; i = (i & (i + 1)) - 1)
      res += t1[i] + t2[i] * (i - (i & (i + 1)) + 1);
    for (int i = r | (r + 1); i < n; i |= i + 1)
      res += t2[i] * (r - (i & (i + 1)) + 1);
    return res;
  }
  void update(int l, int r, long long x) {
    update(l - 1, -x); update(r, x);
  }
  long long get(int l, int r) {
    return get(r) - get(l - 1);
  }
};
```

MoQueries3D.h

d41d8c, 42 lines

```
const int C = 1000;
struct query {
  int l, r, t, it;
  bool operator<(const query& a) {
    if (l / C == a.l / C) {
      if (t / C == a.t / C)
        return r < a.r;
      return t < a.t;
    }
    return l < a.l;
  }
};
struct {
  int l = 1, r = 0, t = 0;
  vector<...> history;

  void add(int pos) {}
  void del(int pos) {}

  void upd() {}
  void upd(int pt) {
    while (pt < t) { //-upd
      int it = --t; upd(hist[it]);
    }
    while (t < pt) { //+upd
      int it = t++; upd(hist[it]);
    }
  }
  int get() {}
  int get(int pl, int pr) {
    while (r < pr)
      add(++r);
    while (l < pl)
      del(l++);
    while (l > pl)
      add(--l);
    while(r > pr)
      del(r--);
    return get();
  }
} mo;
```

Strings (5)

KMP.h

Description: p[i]: abacaba -> 0010123.
Time: $\mathcal{O}(n)$

d41d8c, 9 lines

```
vector<int> p(const string& s) {
  vector<int> p(s.size());
  for (int i = 1; i < s.size(); ++i) {
    int g = p[i - 1];
    while (g && s[i] != s[g]) g = p[g - 1];
    p[i] = g + (s[i] == s[g]);
  }
  return p;
}
```

Zfunc.h

Description: z[i]: abacaba -> 0010301.
Time: $\mathcal{O}(n)$

d41d8c, 12 lines

```
vector<int> z(const string& s) {
  vector<int> z(s.size());
  int l = -1, r = -1;
  for (int i = 1; i < s.size(); ++i) {
    z[i] = i > r ? 0 : min(r - i + 1, z[i - l]);
    while (i + z[i] < s.size() && s[i + z[i]] == s[z[i]])
      ++z[i];
    if (i + z[i] - 1 > r)
      l = i, r = i + z[i] - 1;
  }
  return z;
}
```

Manacher.h

Description: For each position in a string, computes p[0][i] = half length of longest even palindrome around pos i, p[1][i] = longest odd (half rounded down).
Time: $\mathcal{O}(N)$

d41d8c, 13 lines

```
array<vi, 2> manacher(const string& s) {
  int n = sz(s);
  array<vi,2> p = {vi(n+1), vi(n)};
  rep(z,0,2) for (int i=0,l=0,r=0; i < n; i++) {
    int t = r-i+!z;
    if (i<r) p[z][i] = min(t, p[z][l+t]);
    int L = i-p[z][i], R = i+p[z][i]-!z;
    while (L>=1 && R+1<n && s[L-1] == s[R+1])
      p[z][i]++, L--, R++;
    if (R>r) l=L, r=R;
  }
  return p;
}
```

MinRotation.h

Description: Finds the lexicographically smallest rotation of a string.
Usage: `rotate(v.begin(), v.begin()+minRotation(v), v.end());`
Time: $\mathcal{O}(N)$

d41d8c, 8 lines

```
int minRotation(string s) {
  int a=0, N=sz(s); s += s;
  rep(b,0,N) rep(k,0,N) {
    if (a+k == b || s[a+k] < s[b+k]) {b += max(0, k-1); break;}
    if (s[a+k] > s[b+k]) { a = b; break; }
  }
  return a;
}
```

SuffixArray.h

Description: Builds suffix array for a string. `sa[i]` is the starting index of the suffix which is i'th in the sorted suffix array. The returned vector is of size $n+1$, and `sa[0] = n`. The `lcp` array contains longest common prefixes for neighbouring strings in the suffix array: `lcp[i] = lcp(sa[i], sa[i-1])`, `lcp[0] = 0`. The input string must not contain any nul chars.
Time: $\mathcal{O}(n \log n)$

d41d8c, 22 lines

```
struct SuffixArray {
  vi sa, lcp;
  SuffixArray(string s, int lim=256) { // or vector<int>
    s.push_back(0); int n = sz(s), k = 0, a, b;
    vi x(all(s)), y(n), ws(max(n, lim));
    sa = lcp = y, iota(all(sa), 0);
    for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p) {
      p = j, iota(all(y), n - j);
      rep(i,0,n) if (sa[i] >= j) y[p++] = sa[i] - j;
      fill(all(ws), 0);
      rep(i,0,n) ws[x[i]]++;
      rep(i,1,lim) ws[i] += ws[i - 1];
      for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
      swap(x, y), p = 1, x[sa[0]] = 0;
      rep(i,1,n) a = sa[i - 1], b = sa[i], x[b] =
        (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p++;
    }
    for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
      for (k && k--, j = sa[x[i] - 1];
          s[i + k] == s[j + k]; k++);
  }
};
```

SuffixTree.h

Description: Ukkonen's algorithm for online suffix tree construction. Each node contains indices [l, r) into the string, and a list of child nodes. Suffixes are given by traversals of this tree, joining [l, r) substrings. The root is 0 (has l = -1, r = 0), non-existent children are -1. To get a complete tree, append a dummy symbol – otherwise it may contain an incomplete path (still useful for substring matching, though).
Time: $\mathcal{O}(26N)$

d41d8c, 50 lines

```
struct SuffixTree {
  enum { N = 200010, ALPHA = 26 }; // N ~ 2*maxlen+10
  int toi(char c) { return c - 'a'; }
  string a; // v = cur node, q = cur position
  int t[N][ALPHA],l[N],r[N],p[N],s[N],v=0,q=0,m=2;

  void ukkadd(int i, int c) { suff:
    if (r[v]<=q) {
      if (t[v][c]==-1) { t[v][c]=m;  l[m]=i;
        p[m++]=v; v=s[v]; q=r[v];  goto suff; }
      v=t[v][c]; q=l[v];
    }
    if (q==-1 || c==toi(a[q])) q++; else {
      l[m+1]=i;  p[m+1]=m;  l[m]=l[v];  r[m]=q;
      p[m]=p[v];  t[m][c]=m+1;  t[m][toi(a[q])]=v;
      l[v]=q;  p[v]=m;  t[p[m]][toi(a[l[m]])]=m;
      v=s[p[m]];  q=l[m];
      while (q<r[m]) { v=t[v][toi(a[q])];  q+=r[v]-l[v]; }
      if (q==r[m])  s[m]=v;  else s[m]=m+2;
      q=r[v]-(q-r[m]);  m+=2;  goto suff;
    }
  }

  SuffixTree(string a) : a(a) {
    fill(r,r+N,sz(a));
    memset(s, 0, sizeof s);
    memset(t, -1, sizeof t);
    fill(t[1],t[1]+ALPHA,0);
    s[0] = 1; l[0] = l[1] = -1; r[0] = r[1] = p[0] = p[1] = 0;
    rep(i,0,sz(a)) ukkadd(i, toi(a[i]));
  }

  // example: find longest common substring (uses ALPHA = 28)
  pii best;
  int lcs(int node, int i1, int i2, int olen) {
    if (l[node] <= i1 && i1 < r[node]) return 1;
    if (l[node] <= i2 && i2 < r[node]) return 2;
    int mask = 0, len = node ? olen + (r[node] - l[node]) : 0;
    rep(c,0,ALPHA) if (t[node][c] != -1)
      mask |= lcs(t[node][c], i1, i2, len);
    if (mask == 3)
      best = max(best, {len, r[node] - len});
    return mask;
  }
  static pii LCS(string s, string t) {
    SuffixTree st(s + (char)('z' + 1) + t + (char)('z' + 2));
    st.lcs(0, sz(s), sz(s) + 1 + sz(t), 0);
    return st.best;
  }
};
```

Hashing.h

Description: Self-explanatory methods for string hashing.

d41d8c, 44 lines

```
// Arithmetic mod 2^64-1. 2x slower than mod 2^64 and more
// code, but works on evil test data (e.g. Thue-Morse, where
// ABBA... and BAAB... of length 2^10 hash the same mod 2^64).
// "typedef ull H;" instead if you think test data is random,
// or work mod 10^9+7 if the Birthday paradox is not a problem.
typedef uint64_t ull;
struct H {
  ull x; H(ull x=0) : x(x) {}
  H operator+(H o) { return x + o.x + (x + o.x < x); }
  H operator-(H o) { return *this + ~o.x; }
  H operator*(H o) { auto m = (__uint128_t)x * o.x;
    return H((ull)m) + (ull)(m >> 64); }
  ull get() const { return x + !~x; }
  bool operator==(H o) const { return get() == o.get(); }
  bool operator<(H o) const { return get() < o.get(); }
};
static const H C = (ll)1e11+3; // (order ~ 3e9; random also ok)

struct HashInterval {
  vector<H> ha, pw;
  HashInterval(string& str) : ha(sz(str)+1), pw(ha) {
    pw[0] = 1;
    rep(i,0,sz(str))
      ha[i+1] = ha[i] * C + str[i],
      pw[i+1] = pw[i] * C;
  }
  H hashInterval(int a, int b) { // hash [a, b)
    return ha[b] - ha[a] * pw[b - a];
  }
};
```

```
vector<H> getHashes(string& str, int length) {
  if (sz(str) < length) return {};
  H h = 0, pw = 1;
  rep(i,0,length)
    h = h * C + str[i], pw = pw * C;
  vector<H> ret = {h};
  rep(i,length,sz(str)) {
    ret.push_back(h = h * C + str[i] - pw * str[i-length]);
  }
  return ret;
}

H hashString(string& s){H h{}; for(char c:s) h=h*C+c;return h;}
```

OurDoubleHash.h

<bits/stdc++.h> d41d8c, 65 lines

```
using namespace std;
#define ll long long
const int MAXN = 1e6 + 5;

const ll MOD1 = 131'807'699;
const ll MOD2 = 1e9 + 9;
const ll base = 157;

ll expb1[MAXN], expb2[MAXN];

#warning "Call precalc() before use StringHash"
void precalc(){
  expb1[0] = expb2[0] = 1;

  for(int i=1;i<MAXN;i++)
    expb1[i] = expb1[i-1]*base % MOD1,
    expb2[i] = expb2[i-1]*base % MOD2;
}

struct StringHash{
  vector<pair<ll,ll>> hsh;
  string s; // comment S if you dont need it

  StringHash(string& s) : s(s){
    hsh.assign(s.size()+1, {0,0});

    for (int i=0;i<s.size();i++)
      hsh[i+1].first  = ( hsh[i].first *base % MOD1 + s[i] ) %
          MOD1,
      hsh[i+1].second = ( hsh[i].second*base % MOD2 + s[i] ) %
          MOD2;
  }

  ll gethash(int a,int b){
    ll h1 = (MOD1+ hsh[b+1].first  - hsh[a].first *expb1[b-a+1]
        % MOD1) % MOD1;
    ll h2 = (MOD2+ hsh[b+1].second - hsh[a].second*expb2[b-a+1]
        % MOD2) % MOD2;
    return (h1<<32) | h2;
  }
};

// OPTIONAL
int firstDiff(StringHash& a, int la, int ra, StringHash& b, int
    lb, int rb){
  int l=0, r=min(ra-la, rb-lb), diff=r+1;
  while(l <= r){
    int m = (l+r)/2;
    if(a.gethash(la, la+m) == b.gethash(lb, lb+m)) l = m+1;
    else r = m-1, diff = m;
  }
  return diff;
}
```

```
int hshComp(StringHash& a, int la, int ra, StringHash& b, int
     lb, int rb){
  int diff = firstDiff(a, la, ra, b, lb, rb);
  if(diff > ra-la && ra-la == rb-lb) return 0; //equal
  if(diff > ra-la || diff  >  rb-lb) return ra-la < rb-lb ? -2
       : +2; //prefix of the other
  return a.s[la+diff] < b.s[lb+diff] ? -1 : +1;
}

/*LATEX_DESC_BEGIN**************************
**String Hash** - Double Hash
precalc()     -> O(N)
StringHash() -> O(|S|)
gethash()     -> O(1)

StringHash hash(s); -> Cria o Hash da string s
hash.gethash(l, r); -> Hash [L,R] (0-Indexado)
**************************LATEX_DESC_END*/
```

AhoCorasick.h

Description: Aho-Corasick automaton, used for multiple pattern matching. Initialize with AhoCorasick ac(patterns); the automaton start node will be at index 0. find(word) returns for each position the index of the longest word that ends there, or -1 if none. findAll(−, word) finds all words (up to $N\sqrt{N}$ many if no duplicate patterns) that start at each position (shortest first). Duplicate patterns are allowed; empty patterns are not. To find the longest words that start at each position, reverse all input. For large alphabets, split each symbol into chunks, with sentinel bits for symbol boundaries.
Time: construction takes $\mathcal{O}(26N)$, where N = sum of length of patterns. find(x) is $\mathcal{O}(N)$, where N = length of x. findAll is $\mathcal{O}(NM)$. d41d8c, 66 lines

```
struct AhoCorasick {
  enum {alpha = 26, first = 'A'}; // change this!
  struct Node {
    // (nmatches is optional)
    int back, next[alpha], start = -1, end = -1, nmatches = 0;
    Node(int v) { memset(next, v, sizeof(next)); }
  };
  vector<Node> N;
  vi backp;
  void insert(string& s, int j) {
    assert(!s.empty());
    int n = 0;
    for (char c : s) {
      int& m = N[n].next[c - first];
      if (m == -1) { n = m = sz(N); N.emplace_back(-1); }
      else n = m;
    }
    if (N[n].end == -1) N[n].start = j;
    backp.push_back(N[n].end);
    N[n].end = j;
    N[n].nmatches++;
  }
  AhoCorasick(vector<string>& pat) : N(1, -1) {
    rep(i,0,sz(pat)) insert(pat[i], i);
    N[0].back = sz(N);
    N.emplace_back(0);

    queue<int> q;
    for (q.push(0); !q.empty(); q.pop()) {
      int n = q.front(), prev = N[n].back;
      rep(i,0,alpha) {
        int &ed = N[n].next[i], y = N[prev].next[i];
        if (ed == -1) ed = y;
        else {
          N[ed].back = y;
          (N[ed].end == -1 ? N[ed].end : backp[N[ed].start])
            = N[y].end;
          N[ed].nmatches += N[y].nmatches;
          q.push(ed);
        }
      }
    }
  }
  vi find(string word) {
    int n = 0;
    vi res; // ll count = 0;
    for (char c : word) {
      n = N[n].next[c - first];
      res.push_back(N[n].end);
      // count += N[n].nmatches;
    }
    return res;
  }
  vector<vi> findAll(vector<string>& pat, string word) {
    vi r = find(word);
    vector<vi> res(sz(word));
    rep(i,0,sz(word)) {
      int ind = r[i];
      while (ind != -1) {
        res[i - sz(pat[ind]) + 1].push_back(ind);
        ind = backp[ind];
      }
    }
    return res;
  }
};
```